

Version History

Enum: `QCVolumeMode`

Enum: `QCOperatorDeviceMode`

Enum: `QGAISpeakMode`

Class: `QCVolumeInfoModel`

Development Guide

1 Environment configuration

1.1 Add framework

1.2 Configure Bluetooth permissions

1.3 Configure Wi-Fi permissions

2 Usage

2.1 Service UUIDs supported by the device

2.2 import framework library

2.3 Scan Glasses

2.4 Cancel the scan of the Glasses

2.5 Connect the Glasses

2.6 Disconnect

Class: `QCSDKCmdCreator`

Device Mode and Wi-Fi

Video and Audio Configuration

Media Management

Battery and Version Info

DFU Firmware Upgrade Methods

OTA and Other Features

 Error Codes

Version History

Version	Release Date	Change Log
v1.0.0	2025-07-23	Initial release.
v1.0.0	2025-08-12	Add download media resources

Enum: `QCVolumeMode`

Defines volume control modes used across the SDK.

Value	Description
QCVolumeModeMusic	Music volume mode
QCVolumeModeCall	Call volume mode
QCVolumeModeSystem	System volume mode

Enum: QCOperatorDeviceMode

Device operating modes representing the current functional state of the device.

Enum Value	Description
QCOperatorDeviceModeUnkown (0x00)	Unknown mode
QCOperatorDeviceModePhoto (0x01)	Photo mode (taking pictures)
QCOperatorDeviceModeVideo	Video recording mode
QCOperatorDeviceModeVideoStop	Stop video recording
QCOperatorDeviceModeTransfer	Data transfer mode
QCOperatorDeviceModeOTA	OTA (firmware update) mode
QCOperatorDeviceModeAIPhoto	AI-powered photo mode
QCOperatorDeviceModeSpeechRecognition	Speech recognition mode
QCOperatorDeviceModeAuido	Audio recording mode
QCOperatorDeviceModeTransferStop	Stop data transfer (Bluetooth off)
QCOperatorDeviceModeFactoryReset	Factory reset mode
QCOperatorDeviceModeSpeechRecognitionStop	Stop speech recognition
QCOperatorDeviceModeAudioStop	Stop audio recording
QCOperatorDeviceModeFindDevice	Find device mode
QCOperatorDeviceModeRestart	Restart device
QCOperatorDeviceModeNoPowerP2P	Restart P2P without power off
QCOperatorDeviceModeSpeakStart	Voice playback start
QCOperatorDeviceModeSpeakStop	Voice playback stop
QCOperatorDeviceModeTranslateStart	Translation start
QCOperatorDeviceModeTranslateStop	Translation stop

Enum: QGAISpeakMode

AI speaking modes indicating the speaking state of the device.

Enum Value	Description
QGAISpeakModeStart (0x01)	Start speaking
QGAISpeakModeHold	Pause speaking (hold)
QGAISpeakModeStop	Stop speaking
QGAISpeakModeThinkingStart	Start thinking (processing)
QGAISpeakModeThinkingHold	Hold thinking (processing)
QGAISpeakModeThinkingStop	Stop thinking (processing)
QGAISpeakModeNoNet (0xf1)	No network available

Class: QCVolumeInfoModel

Represents volume configuration for music, call, and system modes.

Property	Type	Description
musicMin	NSInteger	Minimum volume for music
musicMax	NSInteger	Maximum volume for music
musicCurrent	NSInteger	Current volume for music
callMin	NSInteger	Minimum volume for calls
callMax	NSInteger	Maximum volume for calls
callCurrent	NSInteger	Current volume for calls
systemMin	NSInteger	Minimum volume for system
systemMax	NSInteger	Maximum volume for system
systemCurrent	NSInteger	Current volume for system
mode	QCVolumeMode	Active volume mode (music/call/system)

Development Guide

1 Environment configuration

1.1 Add framework

Add `QCSDK.framework` to the project, the framework supports iOS 13.0 and above

Note: Because the classification is used in the framework, you need to add settings to the project

Target->Build Settings -> Other Linker Flags add **-ObjC**

1.2 Configure Bluetooth permissions

Configure bluetooth permissions in info.plist file

```
<key>NSBluetoothAlwaysUsageDescription</key>
<string>App needs to use your bluetooth device</string>
<key>NSBluetoothPeripheralUsageDescription</key>
<string>App needs to use your bluetooth device</string>
```

1.3 Configure Wi-Fi permissions

1. Configure local network permissions `NSLocalNetworkUsageDescription` in info.plist file

```
<key>NSLocalNetworkUsageDescription</key>
<string>App needs to use your local network.</string>
```

2. Configure local network permissions `App Transport Security Settings` in info.plist file
3. Configure local network permissions `Bonjour services` in info.plist file
4. Add `Hotspot` in `Signing & Capabilities`

2 Usage

2.1 Service UUIDs supported by the device

Defined in `QCSDKManager.h`, the service UUID supported by the device:

```
extern NSString *const QCSDKSERVERUUID1;
extern NSString *const QCSDKSERVERUUID2;
```

2.2 import framework library

Introduce the framework library into the code

```
#import <QCSDK/QCSDKManager.h>
#import <QCSDK/QCSDKCmdCreator.h>
```

Initialize `[QCSDKManager sharedInstance]` with a singleton

`QCSDKManager`:Peripherals for joining connections

`QCSDKCmdCreator`:Used to send commands to peripherals

2.3 Scan Glasses

initialization

Scanning can only be started when permissions are allowed and Bluetooth is turned on.

Import Apple's CoreBluetooth library and follow two protocols `<CBCentralManagerDelegate,`
`CBPeripheralDelegate>`

```
#import <CoreBluetooth/CoreBluetooth.h>
```

Declare central and peripheral roles

```
/*Central Role,app*/
@property (strong, nonatomic) CBCentralManager *centerManager;

/*Peripheral role, scanned peripherals*/
@property (strong, nonatomic) NSMutableArray<CBPeripheral *> *peripherals;

/*Connected peripheral role*/
@property (strong, nonatomic) CBPeripheral *connectedPeripheral;
```

Instantiate the central role

```
self.centerManager = [[CBCentralManager alloc] initWithDelegate:self queue:nil];
```

Scan Glasses

Using Scan Peripherals

```

NSArray *serviceUUIDStrings = @[QCBANDSDKSERVERUUID1,QCBANDSDKSERVERUUID2];

NSMutableArray *uuids = [NSMutableArray array];
for (id obj in serviceUUIDStrings) {
    if ([obj isKindOfClass:[NSString class]]) {
        CBUUID *uuid = [CBUUID UUIDWithString:obj];
        [uuids addObject:uuid];
    }
}

NSDictionary *option = @{@"CBCentralManagerScanOptionAllowDuplicatesKey" : [NSNumber
numberWithBool:NO]};
[self.centerManager scanForPeripheralsWithServices:uuids options:option];

```

Note: To obtain the scanned peripheral devices in the agent, you can perform secondary filtering through the device name and other related information.

```

- (void)centralManager:(CBCentralManager *)central didDiscoverPeripheral:(CBPeripheral
*)peripheral advertisementData:(NSDictionary<NSString *,id> *)advertisementData RSSI:
(NSNumber *)RSSI {
    if (peripheral.name.length > 0) {
        [self.peripherals addObject:peripheral];
        [self.deviceList reloadData];
    }
}

```

2.4 Cancel the scan of the Glasses

Call the interface of the central role to stop scanning

```

[self.centerManager stopScan];

```

2.5 Connect the Glasses

start connecting

```

self.connectedPeripheral = self.peripherals[indexPath.row];
[self.centerManager connectPeripheral:self.connectedPeripheral options:nil];

```

After the connection is successful, pass in the peripheral device to the SDK

```
- (void)centralManager:(CBCentralManager *)central didConnectPeripheral:(CBPeripheral *)peripheral {
    [[QCSDKManager sharedInstance] addPeripheral:peripheral];
}
```

2.6 Disconnect

```
[self.centerManager cancelPeripheralConnection:self.connectedPeripheral];
```

After disconnecting, remove peripherals

```
- (void)centralManager:(CBCentralManager *)central didDisconnectPeripheral:(CBPeripheral *)peripheral error:(nullable NSError *)error {
    [[QCSDKManager sharedInstance] removePeripheral:peripheral];
}
```

Class: **QCSDKCmdCreator**

QCSDKCmdCreator provides static methods to control the device, manage media, perform firmware updates, and configure various device features.

Device Mode and Wi-Fi

Method	Description
<code>setDeviceMode:success:fail:</code>	Set the device's current operation mode.
<code>openWifiWithMode:success:fail:</code>	Open device Wi-Fi with specified mode, returns SSID and password.

Video and Audio Configuration

Method	Description
<code>setVideoInfo:duration:success:fail:</code>	Set video recording parameters such as angle and duration.
<code>getVideoInfoSuccess:fail:</code>	Retrieve current video recording configuration.
<code>setAudioInfo:duration:success:fail:</code>	Set audio recording parameters such as angle and duration.
<code>getAudioInfoSuccess:fail:</code>	Retrieve current audio recording configuration.

Media Management

Method	Description
<code>getDeviceMedia:fail:</code>	Retrieve counts and sizes of media files on the device.
<code>deleteAllMediasSuccess:fail:</code>	Delete all media files on the device.
<code>deleteMedia:success:fail:</code>	Delete a specified media file by name.
<code>getThumbnail:success:fail:</code>	Retrieve thumbnail image data from a specified media pocket.

Battery and Version Info

Method	Description
<code>getDeviceBattery:fail:</code>	Get battery percentage and charging status.
<code>getDeviceVersionInfoSuccess:fail:</code>	Get hardware and firmware version details.

DFU Firmware Upgrade Methods

Method	Description
<code>switchToDFU:</code>	Switch device to DFU (Device Firmware Update) mode.
<code>initDFUFirmwareType:binFileSize:checksum:crc16:finished:</code>	Initialize DFU process with firmware metadata.
<code>sendFilePacketData:serialNumber:finished:</code>	Send a single packet of firmware data with serial number.
<code>checkMyFirmwareWithData:finished:</code>	Verify sent firmware data.
<code>finishDFU:</code>	Complete the DFU upgrade process.
<code>checkCurrentStatusWithData:finished:</code>	Get current DFU process status from device.
<code>getDFUBandTypeInfoSuccess:fail:</code>	Retrieve DFU band type and status (single/dual band).
<code>switchToOneBandDFU:</code>	Switch to single-band DFU mode.

OTA and Other Features

Method	Description
<code>sendOTAFileLink:finished:</code>	Send OTA firmware download URL to device.
<code>setupDeviceDateTime:</code>	Synchronize device date and time.
<code>sendVoiceHeartbeatWithFinished:</code>	Send heartbeat signal to maintain voice feature connection.
<code>getVoiceWakeupWithFinished:</code>	Get voice wakeup status.
<code>setVoiceWakeup:finished:</code>	Enable or disable voice wakeup feature.
<code>getWearingDetectionWithFinished:</code>	Get wearing detection status.
<code>setWearingDetection:finished:</code>	Enable or disable wearing detection.
<code>getDeviceConfigWithFinished:</code>	Retrieve device configuration.
<code>setAISpeakModel:finished:</code>	Set AI speaking mode.
<code>getVolumeWithFinished:</code>	Get current volume settings.
<code>setVolume:finished:</code>	Set volume settings using <code>QCVolumeInfoModel</code> .
<code>setBTStatus:finished:</code>	Enable or disable Bluetooth.
<code>getBTStatusWithFinished:</code>	Get Bluetooth status.

! Error Codes

Code	Description	Suggested Solution
1000	Unknown error	Check logs and contact support
1001	Initialization failed	Ensure appId/secret is valid and provided
1002	Network unavailable	Check device network status
1003	Timeout	Try again or check server connectivity
1004	Invalid parameters	Verify all required fields are filled
2001	User not logged in	Call <code>login()</code> before using this method
2002	Token expired	Re-authenticate the user
3001	File upload failed	Check file format, size, and permissions
3002	Unsupported operation	Make sure the SDK version supports it